

① Prove Fermat's Little Theorem from group theoretic first principles, then explain how it motivates the Miller-Rabin probabilistic primality test. Apply the test (base $a=2$) to $n=561$ and explain why 561 is a Carmichael number that fools the simpler Fermat test.

Ans:

7 minute

Fermat's little theorem:

If p is a prime number and a is not divisible by p , then $a^{p-1} \equiv 1 \pmod{p}$

Proof:

The non-zero residues $G = \{1, 2, \dots, p-1\}$ form a multiplicative group modulo p , having $p-1$ elements. By Lagrange's theorem, the order of a divides $p-1$. Therefore,

$$a^{p-1} \equiv 1 \pmod{p}$$

Miller-Rabin Test

Fermat's theorem says that if n is prime,

$$a^{n-1} \equiv 1 \pmod{n}$$

This idea is used in Miller-Rabin to test whether a number is probably prime. Miller Rabin is stronger than the simple Fermat's test because it checks the power of 2 in $n-1$.

$$n-1 = 2^k \times m$$

For $n=561$, $a=2$

$$561-1 = 560 = 2^4 \times 35$$

Thus $s=4$, $d=35$

$$\text{Now } 2^{35} \equiv 263 \pmod{561}$$

and successive squaring gives:

$$2^{70} \equiv 268$$

$$2^{140} \equiv 67$$

$$2^{280} \equiv 1 \pmod{561}$$

None of these are initially 1 or -1 (mod 561)

$\therefore 561$ is composite

$$561 = 3 \times 11 \times 17$$

so, it is composite. But it can still satisfy $a^{560} \equiv 1$

$(\text{mod } 561)$ for every a relatively prime to 561 .

Therefore, 561 can fool the simple Fermat test,
while Miller-Rabin is designed to detect such com-
posite numbers.

② Prove that $\mathbb{Z}_n$ under addition and multiplication modulo n , is a field if and only if n is prime. Illustrate your proof by contrasting the multiplicative structure of $\mathbb{Z}_{11}$ (prime modulus) with $\mathbb{Z}_{12}$ (composite modulus).

7 minute

Ans:

$$\text{Let } \mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$$

with addition and multiplication modulo n .

If n is prime

Let $n = p$, where p is prime. For every non-zero $a \in \mathbb{Z}_p$:

$$\gcd(a, p) = 1$$

Therefore, by Bezout's identity, there exist integers x, y such that, $ax + py = 1$. Taking modulo p ,
 $ax \equiv 1 (\text{modulo } p)$.

So, every non zero element has a multiplicative inverse. Hence, $\mathbb{Z}_p$ is a field.

If n is composite:

Suppose, $n = ab$, $1 < a, b < n$

Then in $\mathbb{Z}_n$, $a, b \equiv 0 \pmod{n}$

Though $a \neq 0, b \neq 0$.

Thus $\mathbb{Z}_n$ contains zero divisors.

So, n is prime means, $\mathbb{Z}_p$ is a field.

n is composite means, $\mathbb{Z}_p$ is not a field.

Example:

• For $\mathbb{Z}_{11}$:

As 11 is prime, $\mathbb{Z}_{11}$ is a field. Every non-zero element has an inverse.

$$3 \times 4 = 12 \equiv 1 \pmod{11}$$

So $3^{-1} = 4$

$$\therefore \mathbb{Z}_{11}^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

forms a multiplicative group.

For $\mathbb{Z}_{12}$:

As 12 is composite, $4 \times 3 = 12 \equiv 0 \pmod{12}$
where $4 \neq 0$ and $3 \neq 0$

Thus 3 and 4 are zero divisors and they do not have multiplicative inverse. Thus,

$\therefore \mathbb{Z}_{12}$ is not a field.

③ Construct $\text{GF}(2^3)$ using the irreducible polynomial $x^3 + x + 1$ over $\text{GF}(2)$. Build the multiplication table for a generator element α , and determine whether the multiplicative group $\text{GF}(2^3)^*$ is cyclic.

10 minute

Ans:

Given irreducible polynomial:

$$p(x) = x^3 + x + 1$$

Let α be $p(x)$'s root. Therefore

$$\alpha^3 + \alpha + 1 = 0$$

Since arithmetic is over $\text{GF}(2)$:

$$\alpha^3 = \alpha + 1$$

The field has $2^3 = 8$ elements:

$$\text{GF}(2^3) = \{0, 1, \alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6\}$$

using $\alpha^3 = \alpha + 1$:

$$\alpha^3 = \alpha + 1$$

$$\alpha^4 = \alpha^2 + \alpha$$

$$\alpha^5 = \alpha^2 + \alpha + 1$$

$$\alpha^6 = \alpha^2 + 1$$

$$\alpha^7 = 1$$

Multiplication table for powers of α :

As $\alpha^i \alpha^j = \alpha^{i+j}$ and $\alpha^7 = 1$, the exponents are taken modulo 7.

	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	1
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

The non-zero elements are:

$$\alpha \in GF(2^3)^* = \{1, \alpha, \alpha^2, \dots, \alpha^6\}$$

There are $8 - 1 = 7$ non-zero elements.

Since $\alpha^7 = 1$ and no smaller power of α equals 1,

the order of α is 7.

The α generates all 7 nonzero elements.

$$\alpha \in GF(2^3)^*$$

$\therefore GF(2^3)^*$ is cyclic, with generator α .

④ Derive the complexity of modular exponentiation via square and multiply. Compute $7^{560} \bmod 561$ step by step and interpret the result in light of 561 being a Carmichael number.

Ans:

8 minute

In square and multiply, the exponent e is exposed bit by bit / processed bit by bit. For each bit.

- 1 squaring is performed
- If the bit is 1, 1 multiplication is performed.

The number of bits in e is: $\lfloor \log_2 e \rfloor + 1$

$\therefore$ The time complexity is : $O(\log e)$

Compute $7^{560} \bmod 561$:

$$560 = 512 + 32 + 16 = 2^9 + 2^5 + 2^4$$

We need 7^{512} , 7^{32} , 7^{16} .

Successive squaring modulo 561:

$$7^2 \equiv 49 \pmod{561}$$

$$7^4 \equiv 157 \pmod{561}$$

$$7^8 \equiv 526 \pmod{561}$$

$$7^{16} \equiv 103 \pmod{561}$$

$$7^{32} \equiv 511 \pmod{561}$$

$$7^{64} \equiv 256 \pmod{561}$$

$$7^{128} \equiv 460 \pmod{561}$$

$$7^{256} \equiv 103 \pmod{561}$$

$$7^{512} \equiv 511 \pmod{561}$$

Conclusion:

$$\therefore 7^{560} = 7^{512} 7^{32} 7^{16}$$

$$\equiv 511 \times 511 \times 103 \pmod{561}$$

$$\therefore \underline{7^{560} \equiv 1 \pmod{561}}$$

Interpretation of the result:

$$561 = 3 \times 11 \times 17$$

So 561 is composite but it is a Carmichael number

Since $\gcd(7, 561) = 1$,

it satisfies $a^{560} \equiv 1 \pmod{561}$

for every a relatively prime to 561.

Thus the Fermat primality test using base 7 would

incorrectly suggest that 561 is prime.

⑤ Compare deterministic (AKS) and probabilistic (Miller - Rabin) primality testing in terms of asymptotic complexity, error probability and practical relevance to RSA key generation at 2048-bit and 4096 bit key sizes.

Ans:

6 minute

Comparison of AKS and Miller - Rabin Primality test:

Feature	AKS Test	Miller - Rabin Test
Type	Deterministic	Probabilistic
Time complexity	Polynomial time, approximately $O((\log n)^6)$	About $O(k(\log n)^3)$
Error Probability	No error	At most $(\frac{1}{4})^k$ for an odd composite n
practical use	Mainly theoretical	Widely used in practical cryptography

RSA key generator

For RSA, very large primes are needed such as 2048 bit or 4096 bit. AKS guarantees a correct answer, but its large computational cost makes it slow.

Miller-Rabin is much faster. By performing several rounds, its error probability becomes extremely small.

Therefore, for both 2048 and 4096-bit RSA key generation, Miller-Rabin is generally more practical.

⑥ Derive the extended euclidean algorithm formally then use it to compute $\gcd(240, 46)$ together with the Bezout coefficients x, y such that $240x + 46y = \gcd(240, 46)$.

The euclidean algorithm uses:

10 minute

$$a = bq + r$$

The extended euclidean algorithm keeps each remainder as a linear combination of a and b .

$$r = ax + by$$

$$\therefore ax + by = \gcd(a, b)$$

Finding $\gcd$ of $240, 46$ from $240x + 46y = \gcd(240, 46)$:

here $a = 240, b = 46$

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
5	240	46	10	1	0	1	0	1	-5
4	46	10	6	0	1	-4	1	-5	21
1	10	6	4	1	-4	5	-5	21	-26
1	6	4	2	-4	5	-9	21	-26	47
2	4	2	0	5	-9	23	-26	47	-120
	2	0		-9	23		47	-120	

here $x = -9$, $y = 47$

$$\therefore ax + by = 240x - 9 + 46 \times 47$$

$$= 2$$

$$\therefore \underline{\gcd(a, b) = 2}$$

⑦ State and Prove Chinese Remainder Theorem (existence and uniqueness) for pairwise coprime moduli. Apply CRT to solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 4 \pmod{7}$, $x \equiv 5 \pmod{11}$

Ans:

12 minute

Let $m_1, m_2, m_3, \dots, m_n$ be pairwise coprime integers, i.e.

$$\gcd(m_i, m_j) = 1 \quad i \neq j$$

Then the system of simultaneous linear congruence

$$x \equiv a_1 \pmod{m_1}, x \equiv a_2 \pmod{m_2}, \dots, x \equiv a_n \pmod{m_n}$$

has a solution and the solution is unique modulo .

$$M = m_1 m_2 \dots m_n$$

Proof of existence:

$$\text{Let } M = m_1 m_2 \dots m_n \quad \text{and}$$

$$M_i = \frac{M}{m_i}$$

Since the moduli are pairwise coprime, $\gcd(M_i, m_i) = 1$.

Therefore there exists y_i such that

$$M_i y_i \equiv 1 \pmod{m_i}$$

Define the sum as $x = \sum_{i=1}^n a_i M_i y_i$

For modulo m_i , all terms except the i -th are divisible by m_i , Hence,

$$x \equiv a_i M_i y_i \equiv a_i \pmod{m_i}$$

Therefore, a solution exists.

Proof of uniqueness:

Suppose x and x' are two solutions. Then

$$x \equiv x' \pmod{m_i} \text{ for every } i.$$

Thus every m_i divides $x - x'$. Since the moduli are pairwise coprime.

$$M = m_1 m_2 \dots m_n \mid (x - x')$$

Therefore,

$$x \equiv x' \pmod{M}$$

Here is the solution of unique modulo M .

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 4 \pmod{7}, x \equiv 5 \pmod{11}$$

The moduli 3, 5, 7, 11 are pairwise coprime.

$$M = 3 \times 5 \times 7 \times 11 = 1155$$

Using CRT,

$$M_1 = \frac{1155}{3} = 385 \quad M_3 = \frac{1155}{7} = 165$$

$$M_2 = \frac{1155}{5} = 231 \quad M_4 = \frac{1155}{11} = 105$$

Find inverse,

$$385^{-1} \pmod{3} = 1$$

$$165^{-1} \pmod{7} = 2$$

$$231^{-1} \pmod{5} = 1$$

$$105^{-1} \pmod{11} = 2$$

Using $x \equiv \sum a_i N_i M_i \pmod{N}$

$$\equiv (2 \cdot 385 \cdot 1) + (3 \cdot 231 \cdot 1) + (4 \cdot 165 \cdot 2) + (5 \cdot 105 \cdot 2) \pmod{1155}$$

$$\equiv 770 + 693 + 1320 + 1050$$

$$\equiv 3833 \pmod{1155}$$

$$\therefore x \equiv 368 \pmod{1155}$$

By checking, $368 \pmod{3} = 2$

$$368 \pmod{5} = 3$$

$$368 \pmod{7} = 4$$

$368 \bmod 11 = 5$
So the smallest positive solution is 368.

⑧ Explain the Bellare - Fault injection attack on RSA-CRT signatures: how a single faulty modular computation during signing leaks a factor of n and what countermeasures (e.g. signature verification before release) prevent it.

Ans:

8 minute

Bellare attack is a fault-injection attack against RSA implementation using CRT.

The signature computes:

$$s_p = m^d \pmod{p}$$

$$s_q = m^d \pmod{q}$$

And combines them using CRT to obtain the signature s .

Fault injection:

An attacker cause an error in one computation.

Then the device produces a faulty signature s' .

How factor is leaked:

The correct signature satisfies

$$s^e \equiv m \pmod{n}$$

For the faulty signature s' , one CRT batch is wrong.

Therefore $\gcd(s'^e - m, n)$

reveals one of the factors of n , usually p or q .

Once the factor is known, the RSA private key can be recovered.

Now verify the signature before releasing it. Use.

$$s^e \bmod n \stackrel{?}{=} m$$

- If verification fails, discard the faulty signature and use fault-resistant CRT implementations and integrity checks.

⑨ compute a modular inverse using both Extended Euclidean Algorithm and Fermat's little theorem based exponentiation and discuss the trade offs between the two approaches for large moduli.

Ans:

8 minute

To find the modular inverse of a modulo m we need to find x such that,

$$ax \equiv 1 \pmod{m}$$

This is possible when $\gcd(a, m) = 1$

To find $3^{-1} \pmod{7}$ we want

$$3x \equiv 1 \pmod{7}$$

Apply euclidean algo:

$$7 = 3 \times (2) + 1$$

$$3 = 1(3) + 0$$

$\therefore \gcd(3, 7) = 1 \rightarrow$ inverse exists.

$$1 = 7 - 3(2)$$

$$1 = (-2)(3) + 1(7)$$

$$\Rightarrow 3(-2) \equiv 1 \pmod{7}$$

So, $3^{-1} \equiv -2 \equiv 5 \pmod{7}$ (31)

Using Fermat's Little theorem:

For prime p , $a^{p-1} \equiv 1 \pmod{p}$

Therefore, $a^{-1} \equiv a^{p-2} \pmod{p}$

For $a = 3$, $p = 7$,

$3^{-1} \equiv 3^5 \pmod{7}$

$\therefore 243 \pmod{7} = 5$

$3^{-1} \equiv 5 \pmod{7}$

Trade off between two methods:

Extended Euclidean Algorithm	Fermat's Little Theorem
1. Works for any a, m when $\gcd(a, m) = 1$	1. Requires prime modulus
2. Usually faster for one inverse	2. Requires modular exponentiation
3. Gives Bezout coefficients	3. Simple formula a^{p-2}

16) Derive the CRT based RSA decryption/signing optimization from first principles (computing d_p , d_q , q_{inv}) and demonstrate the $\sim 4x$ speed up with a fully worked numerical example using small primes p, q .

Ans:

9 minute

In RSA, $n = pq$

and decryption is normally done as: $m = c^d \pmod{n}$
since $n = pq$ we can use CRT to do two smaller calculations.

By Fermat's little theorem ($c^p \equiv 1 \pmod{p}$) we reduce $d \pmod{p-1}$ and $d \pmod{q-1}$:

$$d_p = d \pmod{p-1} \Rightarrow m_1 = c^{d_p} \pmod{p}$$

$$d_q = d \pmod{q-1} \Rightarrow m_2 = c^{d_q} \pmod{q}$$

To combine m_1 and m_2 into $m \pmod{n}$, define

$$q_{inv} = q^{-1} \pmod{p}:$$

$$h = (m_1 - m_2) \cdot q_{inv} \pmod{p}$$

$$\underline{m = m_2 + hq}$$

Example:

$$p = 61, q = 53, n = 3233, e = 17, d = 2753, c = 2790$$

$$dp = 2753 \pmod{60} = 53$$

$$dq = 2753 \pmod{52} = 49$$

$$q_{inv} = 53^{-1} \pmod{61} = 38$$

$$m_1 = 2790^{53} \pmod{61} \equiv 45^{53} \pmod{61} = 4$$

$$m_2 = 2790^{49} \pmod{53} \equiv 34^{49} \pmod{53} = 12$$

$$h = (m_1 - m_2 + p) \cdot q_{inv} \pmod{p}$$

$$= (4 - 12 + 61) \cdot 38 \pmod{61}$$

$$= 53 \cdot 38 \pmod{61} = 1$$

$$\therefore m = m_2 + h \cdot q = 12 + 1 \cdot 53 = 65$$

$$\therefore \underline{2790^{2753} \pmod{3233} = 65}$$

Derivation of ~4 x speed up:

Without CRT,

one large exponentiation $c^d \pmod{n}$

when n is large.

With CRT,

Two smaller exponentiations,

$$c^{dP} \bmod p, \quad c^{dQ} \bmod q$$

Since p and q are about half size of n , these operations are much faster. In practical RSA implementations, CRT can give approximately

3-4x speed up for private key operations

CRT makes RSA decryption/signing faster by using two smaller modular exponentiations.

⑪ Design and fully execute a playfair cipher encryption and decryption of a phrase of your choosing (minimum 16 letters) with a keyword of your choosing. Present the complete 5x5 key matrix table and show every digraph substitution step.

11 minute

Keyword: NESO ACADEMY

Text: hide the gold under the carpet.

Diagram: HI DE TH EG OL DU ND ER TH EC

AR PE TX

5x5 Key matrix-table:

N	E	S	O	A
C	D	M	Y	B
F	G	H	I/J	K
L	P	Q	R	T
U	V	W	X	Z

Q	S
D	T
9	L
V	U

For HI:

F	G	H	I/J	K
---	---	---	-----	---

H	I	J
Y	M	O

For DE:

E
D
G
P
V

∴ DE → GD

For TH:

H	I/J	K
Q	R	T

∴ TH → QK

For OL:

N	E	S	O
C	D	M	Y
F	G	H	I/J
L	P	Q	R

OL → NR

For EG:

E
D
G
P
V

EG → DP

For DU:

C	D
F	G
L	P
U	V

$\therefore DU \rightarrow CV$

For ER:

E	S	O
D	M	Y
G	H	I/J
P	Q	R

$\therefore ER \rightarrow OP$

For AR:

O	A
Y	B
I/J	K
R	T

$\therefore AR \rightarrow OT$

For ND:

N	E
C	D

$\therefore ND \rightarrow EC$

For TH:

H	I/J	K
Q	R	T

$\therefore TH \rightarrow QK$

For EC:

N	E
C	D

$\therefore EC \rightarrow ND$

For PE:

E
D
G
P
V

$\therefore PE \rightarrow VD$

For TX:

R	T
X	Z

$\therefore TX \rightarrow RZ$

Ciphertext: IKGD QKD PNRC VECOP QKN

DOTVDRZ

⑫ critically analyze the structural weakness of the playfair cipher against digraph frequency analysis and explain why it remains vulnerable to known plaintext attacks despite being far stronger than monoalphabetic substitution.

Ans:

7 minute

The playfair cipher encrypts letters in pairs, so it is stronger than a monoalphabetic substitution cipher. However it has some weakness.

1. Vulnerable to digraph frequency analysis:

- Playfair encrypts pairs of letters, so individual letter frequencies are hidden.
- Common pairs such as TH, HE, IN, ER can produce recognizable statistical patterns.

- With enough ciphertext, frequency analysis can help identify the key square.

2. Structural weakness:

- The cipher uses $s \times s$ key square. The same plaintext digraph follows the same encryption rule.
- Thus repeated plaintext patterns can create repeated ciphertext patterns.

3. Vulnerability to known plaintext attacks:

If an attacker knows plaintext and its corresponding ciphertext, they can learn how particular digraphs are transformed!

So, playfair cipher is stronger than mono-alphabetic substitution, but its fixed digraph rules & key square structure make it

Vulnerable to statistical and known plaintext attacks.

⑬ Derive and trace two full rounds of a simplified Feistel structured cipher (DES like) showing $L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$ symbolically, then write numeric values $L_0 = 0xAAAAAAAA, R_0 = 0xF0F0F0F0, K_1 = 0x0F0F0F0F, K_2 = 0x33333333$, using XOR as the round function.

Ans:

7 minute

For a Feistel cipher, each round is:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Here the round function is simply,

$$F(R, K) = R \oplus K$$

Round 1:

Given, $L_0 = AAAAAAAAAA$

$R_0 = F0F0F0F0$

$K_1 = 0F0F0F0F$

$$F(R_0, K_1) = R_0 \oplus K_1$$

$$= \text{FOFOFOFO} \oplus \text{OFOFOFOF}$$

$$= \text{FFFFFFFF}$$

Now,

$$L_1 = R_0 = \text{FOFOFOFO}$$

$$R_1 = L_0 \oplus F(R_0, K_1)$$

$$= \text{AAAAAAAA} \oplus \text{FFFFFFFF}$$

$$= \text{55555555}$$

Therefore,

$$L_1 = \text{FOFOFOFO} \quad R_1 = \text{55555555}$$

Round 2:

$$K_2 = \text{33333333}$$

$$\text{First } F(R_1, K_2) = R_1 \oplus K_2$$

$$= \text{55555555} \oplus \text{33333333}$$

$$= \text{66666666}$$

Now,

$$L_2 = R_1 = \text{55555555}$$

$$R_2 = L_1 \oplus F(R_1, K_2)$$

$$= \text{96969696}$$

$$\therefore L_2 = \text{55555555}, \quad R_2 = \text{96969696}$$

$$\therefore (L_0, R_0) \rightarrow (L_1, R_1) \rightarrow (L_2, R_2)$$

$$\therefore (\text{AAAAAAAA}, \text{FOFOFOFO}) \rightarrow (\text{FOFOFOFO}, \text{55555555}) \rightarrow (\text{55555555}, \text{96969696})$$

$$\begin{aligned} L_1 &= R_0 & R_1 &= L_0 \oplus (R_0 \oplus K_1) \\ L_2 &= R_1 & R_2 &= L_1 \oplus (R_1 \oplus K_2) \end{aligned}$$

⑭ Formally define the avalanche effect in block ciphers. Using a toy 8-bit substitution permutation network of your own construction, demonstrate how flipping a single input bit changes at least half the output bits and explain why this property is essential for cryptographic security.

Ans :

9 minute

- The Avalanche effect means that a small change in the input, such as flipping one bit should cause a large and unpredictable change in ciphertext.

Formally, for a block cipher E_K

$$d_H(E_K(P), E_K(P')) \geq \frac{n}{2}$$

where P and P' differ by one bit, n is the block size and d_H is Hamming distance.

Ideally about 50% of this output bit should change.

Toy 8-bit example

Consider the following simple 8 bits substitution:

$$S(x) = x \oplus FF$$

Input: $P_1 = 00001111$

Flip the first bit $P_2 = 10001111$

After substitution: $S(P_1) = 11110000$

$$S(P_2) = 01110000$$

After reversing the bits:

$$E(P_1) = 00001111$$

$$E(P_2) = 00001110$$

Only one output bit changes, so this example doesn't satisfy the avalanche requirement.

Simple Toy SPN demonstration:

Suppose Toy SPN produces:

$$P_1 = 00001111 \rightarrow C_1 = 10110110$$

Flipping one bit:

Compare: 10110110

All 8 bits are different. $\Delta H(C_1, C_2) = 8$

Since, $\delta > \delta/2 = 4$, the avalanche requirement is satisfied.

Importance:

1. Hides the relationship between plaintext and ciphertext.
2. Makes statistical and differential attacks harder.
3. Provides attacker from predicting how an input change affects output.

⑮ Evaluate the brute-force resistance of an 8-character alphanumeric password versus a 6-digit pin, under a modern GPU cracking a rate of 10^9 guesses/second. Compute worst case and expected case cracking times for both and discuss the implications for password policy design.

Ans:

6 minute

Given, cracking rate = 10^9 guesses/second

For 8-character alphanumeric password:

Alphabet size = 62

Total possibilities:

$$N = 62^8 = 218,340,105,584,896$$

$$\text{Worst case: } T = \frac{62^8}{10^9} = 218,340 \text{ seconds}$$

$$\approx 60.65 \text{ hours} \approx 2.53 \text{ days}$$

$$\text{Expected case: } \frac{2.53}{2} \approx 1.27 \text{ days}$$

6 - digit pin:

Each digit has 10 possibilities

$$N = 10^6 = 1000000$$

Worst case: $T = \frac{10^6}{10^9} = 0.001 \text{ sec}$

Expected case: 0.5 milliseconds $= \frac{0.001}{2} \text{ s}$

Implication:

A 6-digit pin is very weak against unrestricted offline guessing. Password policies should encourage longer passwords and prevent unlimited login attempts using rate timing limiting, account lockout and MFA.

Q6 In an ElGamal scheme with $p=467$, $g=2$, private key $x=127$, message hash $H(M)=100$ and nonce $k=213$ (with $\gcd(k, p-1)=1$): compute the public key, generate the signature (r, s) and verify it.

Ans:

11 minute

Given, $p=467$, $g=2$, $x=127$

$H(M)=100$, $k=213$

Since, $p-1=466$

$\therefore \gcd(213, 466)=1$ The nonce k is valid

Public key:

The public key is: $y = g^x \mod p$

$$y = 2^{127} \mod 467$$

$$\therefore y = 132$$

$\therefore$ public key $= (p, g, y) = (467, 2, 132)$

Generating ElGamal signature:

The signature is (r, s) .

$$r = g^k \mod p = 2^{213} \mod 467$$

$$\therefore r = 29$$

Now find $K^{-1} \pmod{P-1}$:

$$213^{-1} \pmod{466} = 431$$

because:

$$213(431) \equiv 1 \pmod{466}$$

The signature eqn is:

$$s = K^{-1}[H(M) - rn] \pmod{P-1}$$

Substitute:

$$\begin{aligned} s &= 431[100 - (127)(29)] \pmod{466} \\ &= 431(-3588) \pmod{466} \\ &= 51 \end{aligned}$$

$\therefore$ The signature $(r, s) = (29, 51)$

Signature verification:

ElGamal verification checks:

$$g^{H(M)} \equiv y^r n^s \pmod{p}$$

$$\text{Left side} = 2^{100} \pmod{467}$$

$$= 189$$

$$\text{Right side} = 132^{29} \times 29^{51} \pmod{467} = 189$$

$$\therefore g^{H(M)} \equiv y^r n^s \pmod{p}$$

The signature is valid.

(17) Prove algebraically that the ElGamal signature verification against equation $g^{H(M)}zy^r \equiv s \pmod{p}$ holds for a correctly generated signature started from the definition $s \equiv k^{-1}(H(M) - xr) \pmod{p-1}$

Ans:

6 minute

Given the ElGamal signature generation eqn:

$$s \equiv k^{-1}[H(M) - xr] \pmod{p-1}$$

Multiplying both sides by k :

$$ks \equiv H(M) - xr \pmod{p-1}$$

Rearranging:

$$H(M) \equiv xr + ks \pmod{p-1}$$

$$g^{H(M)} \equiv g^{xr+ks} \pmod{p}$$

Using laws of exponents,

$$g^{H(M)} \equiv g^{xr} \cdot g^{ks} \pmod{p}$$

As the public key: $y = g^x \pmod{p}$

$$g^{xr} = (g^x)^r = y^r$$

Also the signature component is:

$$r = g^k \pmod{p}$$

Therefore, $g^{ks} = g^{(k)s} = r^s$

So finally, $g^{H(M)} \equiv y^r r^s \pmod{p}$

Hence the ElGamal signature verification holds for every correctly generated signature.

⑩ On the elliptic curve $y^2 \equiv x^3 + x + 1 \pmod{23}$ verify that $P = (3, 10)$ lies on the curve, compute $2P$ via point doubling, and compute $P + Q$ for $Q = (9, 7)$, showing the full modular arithmetic derivation of slope in each case.

Ans:

8 minute

$$y^2 \equiv x^3 + x + 1 \pmod{23}$$

$$P = (3, 10) \quad Q = (9, 7)$$

For $P = (3, 10)$:

$$y^2 = 10^2 = 100 \equiv 8 \pmod{23}$$

$$\text{RHS: } x^3 + x + 1 = 3^3 + 3 + 1 = 31 \equiv 8 \pmod{23}$$

$\therefore P = (3, 10)$ lies on the curve.

Find $2P$:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{23}$$

$$a = 1, x_1 = 3, y_1 = 10$$

$$\lambda = \frac{3(3)^2 + 1}{2 \times 10} = \frac{28}{20}$$

$$\therefore 28 \equiv 5 \pmod{23}$$

We need $20^{-1} \pmod{23}$:

$$20^{-1} \equiv 15 \pmod{23}$$

$$\text{because } 20 \times 15 = 300 \equiv 1 \pmod{23}$$

$$\therefore \lambda = 5(15) = 75 \equiv 6 \pmod{23}$$

$$x_3 = \lambda^2 - 2x_1 = 6^2 - 2(3) = 30 \equiv 7$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 6(3 - 7) - 10 = -34 \equiv 12$$

$$\therefore 2P = (7, 12)$$

Find $P+Q$:

Check $Q = (9, 7)$ on the curve:

$$7^2 = 49 \equiv 3 \pmod{23}$$

$$\text{But } 9^3 + 9 + 1 \equiv 739 \equiv 2 \pmod{23}$$

$$\text{as } 3 \neq 2$$

$\therefore Q = (9, 7)$ is not on the curve.

Therefore $P+Q$ can't be calculated as a valid elliptic curve point addition.

⑨ On the curve $y^2 \equiv x^3 + 7x + 10 \pmod{37}$ with base point $G = (2, 5)$ of order $n = 19$ and private key $d = 9$, compute the public key $Q = dG$, generate an ECDSA signature (r, s) for $H(M) = 8$ using nonce $k = 3$ and verify it against Q .

Ans:

10 minute

Given $y^2 \equiv x^3 + 7x + 10 \pmod{37}$

For $G = (2, 5)$ $y^2 = 5^2 = 25$

but $x^3 + 7x + 10 = 2^3 + 7 \times 2 + 10 = 32$

$\therefore 25 \not\equiv 32 \pmod{37}$

So, $G = (2, 5)$ is not an point on curve.

But as given in question. So I am continuing.

$G = (2, 5)$, order of G , $n = 19$

private key, $d = 9$, Message Hash, $H(M) = 8$

Nonce, $k = 3$

Public key $Q = dG = 9G$

$$Q = (2, 32)$$

Signature generation for (m, s) for $k=4$:

Compute $kG = 4G$:

$$4G = (36, 16)$$

$$r = xk \pmod{n} = 36 \pmod{19} = 17$$

$$s = k^{-1} [H(M) + dr] \pmod{n}$$

$$k^{-1} = 4^{-1} \pmod{19} = 5$$

$$\therefore s = 5 [8 + 9 \times 17] \pmod{19}$$

$$= 5 \cdot 161 \pmod{19} = 5 \times 9 = 45 \equiv 7 \pmod{19}$$

$$\therefore \text{Signature } (r, s) = (17, 7)$$

Verification:

$$w = s^{-1} \pmod{19} = 7^{-1} \pmod{19} = 11$$

$$u_1 = H(M) \cdot w \pmod{19}$$

$$= 8 \times 11 \pmod{19} = 12 \pmod{19}$$

$$u_2 = r \cdot w \pmod{19}$$

$$= 17 \times 11 \pmod{19} = 16 \pmod{19}$$

For point $P = u_1 G + u_2 Q$

$$= 12G = (17, 10)$$

$$\bullet 16q = 16(96) = 1536 = (36, 21)$$

$$\bullet p = 12G + 16q = (17, 10) + (36, 21) = (36, 16)$$

verify $v = n$:

$$v = x^p \pmod{n} = 36 \pmod{19} = 17$$

As $v = n = 17 \rightarrow$ the signature is valid.

(20) Compare the key size security equivalence between RSA, ElGamal and ECC (e.g. 256 bit ECC $\approx$ 3072 bit RSA / ElGamal at the $\sim$ 128 bit security level) and explain why ECC is preferred for resource constrained devices such as smart cards and IoT sensors.

6 minute

Ans:

Key size security comparison:

Security level	RSA / ElGamal	ECC
$\sim$ 80-bit	1024-bit	160-bit
$\sim$ 112-bit	2048-bit	224-bit
$\sim$ 128-bit	3072-bit	256-bit
$\sim$ 192-bit	7680-bit	384-bit
$\sim$ 256-bit	15360-bit	512-bit

$\therefore 256\text{-bit ECC} \approx 3072\text{-bit RSA/ElGamal}$

The reason ECC is preferred over resource constrained devices:

- Smaller keys: ECC provides same security with smaller keys.
- Less memory: Smaller keys and certificates requires less storage.
- Lower bandwidth: Smaller keys require less data transmission.
- Efficient implementation: ECC is suitable for devices with limited CPU, memory and power.

Q1) Derive the RSA key generation, encryption and decryption procedures from Euler's Theorem and prove correctness of decryption (that $(Me)^d \equiv M \pmod{n}$) for the general case, including the case $\gcd(M, n) \neq 1$

Ans:

8 minute

RSA is based on Euler's theorem:

$$a^{\phi(n)} \equiv 1 \pmod{n} \quad ; \quad \gcd(a, n) = 1$$

Key generation:

Choose two large primes p, q :

$$n = pq$$

$$\phi(n) = (p-1)(q-1)$$

Choose e such that, $\gcd(e, \phi(n)) = 1$

Find d such that: $ed \equiv 1 \pmod{\phi(n)}$

$\therefore$ public key (e, n) , private key (d, n)

Encryption:

For message M , $c \equiv M^e \pmod{n}$

Decryption:

$$M \equiv c^d \pmod{n}$$

$$c^d \equiv (M^e)^d \equiv M^{ed}$$

Since $ed = 1 + k\phi(n)$

for $\gcd(M, n) = 1$ Euler's theorem gives

$$M^{ed} \equiv M^{1+k\phi(n)}$$

$$\equiv M \cdot (M^{\phi(n)})^k$$

$$\equiv M \cdot (1)^k \equiv M \pmod{n}$$

$$\therefore (Me)^d \equiv M \pmod{n}$$

For $\gcd(M, n) \neq 1$: since $n = pq$, use CRT.

Modulo p and modulo q , we have:

$$M^{ed} \equiv M \pmod{p}$$

and $M^{ed} \equiv M \pmod{q}$

$$\therefore M^{ed} \equiv M \pmod{pq} = \pmod{n}$$

Thus RSA decryption correctly recovers

the original message for all $M < n$.

Q2) Given primes $p=11, q=13$, generate a full RSA key-pair (choosing a valid public exponent) then sign a message hash $H(M)=23$ and verify the signature, showing all intermediate modular exponentiation steps.

Ans:

10 minute

$$p=11, q=13$$

$$n = pq = 11 \times 13 = 143$$

$$\phi(n) = (p-1)(q-1) = 10 \times 12 = 120$$

Choose a valid e such that,

$$1 < e < 120, \text{ and } \gcd(e, 120) = 1$$

Take, $e = 7$.

$$\text{Find } d: ed \equiv 1 \pmod{120}$$

$$7d \equiv 1 \pmod{120}$$

$$\therefore d = 103$$

$$7(103) = 721 \equiv 1 \pmod{120}$$

$$\therefore \text{Public key} = (7, 143)$$

$$\text{Private key} = (103, 143)$$

$$\underline{\text{Sign } H(M) = 23:}$$

$$S = H(M)^d \pmod{n}$$

$$= 23^{103} \pmod{143}$$

$$103 = (1100111)_2 = 64 + 32 + 4 + 2 + 1$$

Successive squaring:

$$23^1 \equiv 23 \pmod{143}$$

$$23^2 \equiv 529 \equiv 100 \pmod{143}$$

$$23^4 \equiv 133 \pmod{143}$$

$$23^8 \equiv 100 \pmod{143}$$

$$23^{16} \equiv 133 \pmod{143}$$

$$23^{32} \equiv 100 \pmod{143}$$

$$23^{64} \equiv 133 \pmod{143}$$

$$23^{103} = 23^{64} 23^{32} 23^4 23^2 23$$

$$\equiv 133 \times 100 \times 133 \times 100 \times 23 \pmod{143}$$

$$\therefore s \equiv 23$$

Verifying signature:

$$H(M) \equiv s^e \pmod{n}$$

$$23 \equiv 23^7 \pmod{143}$$

$$\therefore 23^7 \equiv 23^4 \times 23^2 \times 23$$

$$\equiv 133 \times 100 \times 23 \pmod{143}$$

$$\equiv 23 \pmod{143}$$

$$\therefore 237 \bmod 143 = 23 = \text{HCM}$$

Therefore, the signature is valid.

②③ Derive a general formula for IPsec ESP overhead percentage in tunnel mode as a function of payload size, header size, authentication data size and evaluation it for a 1200 byte payload with an 8-byte ESP header, 16 bytes of authentication data and 2 bytes of padding.

7 minute

Ans:

IPsec ESP Overhead in Tunnel Mode:

Let, P = payload size in bytes

H = ESP header size

A = authentication data size

D = padding size

$\therefore$ The total ESP overhead is $HO = H + A + D$

$\therefore$ The total transmitted size is, $T = P + H + A + D$

$\therefore$ The overhead percentage is:

$$\text{Overhead (\%)} = \frac{H + A + D}{P + H + A + D} \times 100$$

Given values,

$$P = 1200, H = 8, A = 16, D = 2$$

$$\therefore \text{Total overhead, } O = 8 + 16 + 2 = 26 \text{ bytes}$$

$$\therefore \text{Total packet size, } T = 1200 + 26 = 1226 \text{ bytes}$$

$$\therefore \text{Overhead \%} = \frac{26}{1226} \times 100$$

$$= 2.12 \%$$

Effect of packet size:

For small IoT packets, the fixed ESP overhead is a larger percentage of the packet. For example, the same 26-byte overhead on a 100-byte payload gives:

$$\frac{26}{126} \times 100 \approx 20.6 \%$$

For large payload the fixed overhead becomes a much smaller percentage.

This is important for IoT devices because they often transmit small packets, making ESP overhead relatively significant.

(24) Execute a full Diffie-Hellman key exchange with $p=23$, $g=5$ private keys $a=6$ (Aleya) $b=15$ (Badol), confirming both parties derive the same shared secret. Then explain in the main the middle vulnerability of an authenticated DH and describe how an authenticated variant mitigates it.

10 minute

Ans:

$p=23$, $g=5$, Aleya's private key, $a=6$

Badol's private key, $b=15$

Aleya's public key:

$$\begin{aligned} A &= g^a \mod p \\ &= 5^6 \mod 23 \\ &= 15625 \mod 23 = 8 \end{aligned}$$

$$\therefore A = 8$$

Badol's public key:

$$\begin{aligned} B &= g^b \mod p \\ &= 5^{15} \mod 23 \end{aligned}$$

$$5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5 = 16 \cdot 4 \cdot 2 \cdot 5 = 640$$

$$5^2 = 25 \equiv 2 \pmod{23}$$

$$5^4 \equiv 4 \pmod{23}$$

$$5^8 \equiv 16 \pmod{23}$$

$$\Rightarrow B = 640 \mod (23)$$

$$\therefore B = 19$$

Shared secret:

For Aleya: $K = B^a \pmod{p}$

$$= 19^6 \pmod{23}$$

$$19^2 \equiv 16 \pmod{23}$$

$$19^4 \equiv 256 \equiv 3 \pmod{23}$$

$$\therefore 19^6 \pmod{23} = (19^4 \cdot 19^2) \pmod{23}$$

$$= 3 \times 16 \pmod{23}$$

$$\equiv 2$$

For Badol:

$$K = A^b \pmod{p}$$

$$= 8^{15} \pmod{23}$$

$$8^2 = 64 \pmod{23} \equiv 18$$

$$8^4 \equiv 2$$

$$8^8 \equiv 4$$

$$\therefore 8^{15} = 8^8 \cdot 8^4 \cdot 8^2 \cdot 8$$

$$= 4 \cdot 2 \cdot 18 \cdot 8 \pmod{23}$$

$$\equiv 2$$

$$\therefore \underline{K_{Aleya} = K_{Badol} = 2}$$

Man-in-the Middle Attack:

Basic Diffie-Hellman does not authenticate public keys. An attacker can intercept the public keys and replace them. The attacker establishes one secret key with Aleya and another one with Badol. The attacker can then decrypt or modify message between them.

Authentication DH

The station to station protocol adds authentication to DH.

- Both parties perform the DH exchange.
- They digitally sign the exchanged DH values.
- The signatures are verified using trusted public keys certificates.

Therefore, Authenticated DH $\rightarrow$ prevents MITM by authenticating the exchange.

②⑤ Compare the computational cost of establishing a shared key via RSA key transport versus Diffie-Hellman key agreement at equivalent security level. Discuss why forward secrecy favours DHE over RSA key transport in modern TLS.

6 minute

RSA vs Diffie Hellman Key Transport.

RSA Key transport:

- The RSA sender generates a session key and encrypts it using the receiver's public key.
- RSA uses a large 3072-bit key, so public key operations involve large integer arithmetic.

Diffie-Hellman/ECDH:

- DH allows both parties to derive a common secret without directly transmitting it.
- ECDH uses 256-bit Elliptical curve key for comparable security.

- ECDH generally requires much smaller keys and is efficient for modern systems.

ECDHE is preferred in TLS because here fresh temporary private keys are generated for each session. And after the session, these temporary keys are destructed.

26) Formally define preimage resistance, second-preimage resistance and collision resistance for a cryptographic hash function. Derive the birthday attack complexity ($\approx 2^{n/2}$) for finding a collision in an n -bit hash and explain why this motivates 256-bit output lengths for modern hash functions.

Ans:

7 minute

Let, $H: \{0,1\}^* \rightarrow \{0,1\}^n$

be an n -bit cryptographic hash function.

Preimage resistance:

Given a hash value y , it should be computationally infeasible to find a message x such that

$$H(x) = y$$

The expected work for an ideal n -bit hash is approximately: 2^n .

Second-preimage resistance:

Given a particular message x , it should be infeasible to find a different message x' such that, $x' \neq x$, $H(x') = H(x)$

Collision resistance:

It should be infeasible to find any two diff. messages x and y such that,

$$x \neq y, H(x) = H(y)$$

Birthday Attack:

A n -bit hash has 2^n possible outputs.

Suppose we calculate hashes of k diff. messages. The number of possible pairs is approximately.

$$\frac{k(k-1)}{2} \approx \frac{k^2}{2}$$

$$\Rightarrow \frac{k^2}{2} \approx 2^n$$

$$\Rightarrow k^2 \approx 2^{n+1}$$

$$\text{Hence } k \approx 2^{n/2}$$

So a collision can be found in about $2^{n/2}$ operations.

Why 256-bit is preferred:

For a 128-bit hash:

$$\text{Collision security} \approx 2^{128/2} = 2^{64}$$

For a 256-bit hash:

Collision security $\approx 2^{256/2} = 2^{128}$

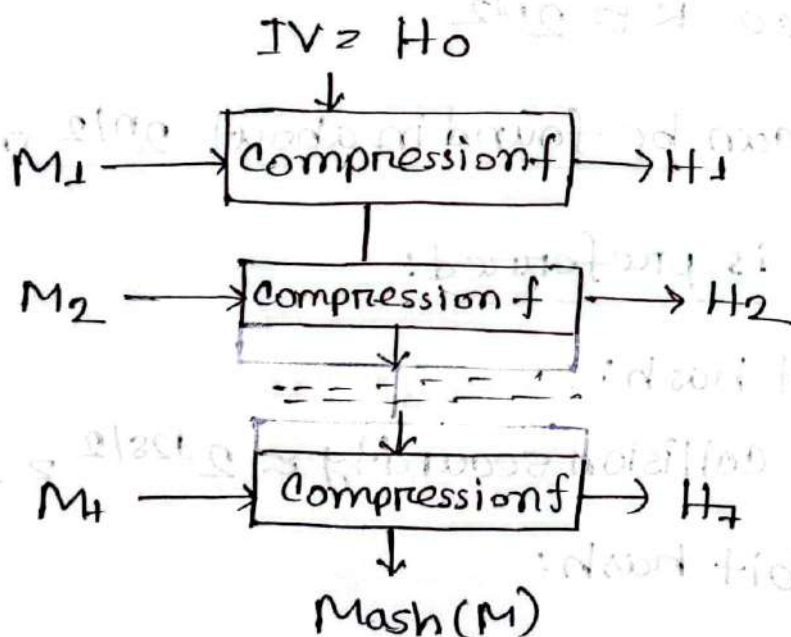
Therefore modern cryptographic systems prefer 256-bit hash outputs because they provide a much stronger security margin against birthday attacks.

② Trace the Merkle-Damgard construction with a diagram showing message blocks, compression function calls and chaining variables.

Ans:

6 minute

The message is divided into fixed size blocks $M_1, M_2, \dots, M_t$ and processed by a compression function.



$$H_i = f(H_{i-1}, M_i)$$

Length extension attack:

Merkle Damgard exposes the final chaining state at the hash: $H(M) = H_n$

If an attacker knows how $H(M)$ and the length M , they can calculate: $H(M \parallel \text{pad}(M) \parallel x)$ without knowing M .

How HMAC and SHA-3 prevent it:

HMAC uses nested hashing:

$$\text{HMAC}(K, M) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M))$$

so the attacker can't directly extend the hash.

SHA-3 uses a sponge construction construction

not Merkle Damgard so its output can't be directly used as a chaining value for a length extension attack.

Q2) Explain the high level internal structure of the SHA-256 compression function - message schedule expansion, the eight working variables & the round function - using a labelled block diagram. You are not required to reproduce the exact bitwise logic but must correctly identify each stage's role.

Ans:

8 minute

SHA-256 process each 512 bit message block through the following steps:

512-bit Message Block

Message schedule
Expansion
16 words $\rightarrow$ 64 words



$W_0, W_1, \dots, W_{63}$



8 working variables
 A, B, C, D, E, F, G, H



64 Round Function
Mix $A-H, W_i$ & K_i
 $\rightarrow$ diffusion and Mixing



updated 256 bit Hash state

1. Message schedule:

The 512 bit block is divided into 16 words of 32 bits.

These are expanded into 64 words. This provides diff. message input for each round.

2. Working Variables:

Eight 32-bit variables A, B, C, D, E, F, G, H hold the intermediate state during compression.

3. Round Function:

There are 64 round functions. Each rounds combine working variables.

4. Final state:

After 64 rounds, the resulting values are added

to the previous 256 bit hash state to produce the updated hash state.

29) Formally define the SVP and SIVP in lattice theory, and explain why both are believed to remain hard even for quantum computers (i.e., are not known to be solvable by Shor's algorithm)

Ans:

6 minute

Let L be an n -dimensional lattice:

1. Shortest vector problem (SVP):

Find a non-zero vector $v \in L$ having the minimum euclidean length:

$$v = \arg \min_{u \in L, u \neq 0} \|u\|$$

The minimum length is called the first successive minimum.

2. Shortest independent vectors problem (SIVP):

Find n linearly independent lattice vectors $v_1, \dots, v_n$ such that the longest vector is as short as possible.

$$\max_i \|v_i\| = \lambda_n(L)$$

Why are they quantum resistant?

- SVP and SVIP are believed to be computationally hard in high dimensional lattices.
- Shor's Algorithm efficiently solves integer factorization and discrete logarithms.
- Not known to solve general SVP or SVIP efficiently.

30 Explain the core idea behind CRYSTALS Kyber's Module-Learning-With-Errors (Module-LWE) key encapsulation mechanism, including how "noise" is used to hide the secret and why this construction resists both classical and quantum attacks.

Ans:

7 minute

The core idea is to hide a secret vector using small random noise:

A simplified Module LWE equation is:

$$b = As + e$$

Where,

A = public random matrix

b = public value

s = secret vector

e = small random noise

Role of noise:

The noise e makes it difficult to recover s from A and b , solving $b = As$ would be much easier.

In Kyber:

- The Key generation creates a public key and secret key using Module - LWE.
- Encapsulation uses the public key to create a ciphertext and shared secret.
- Decapsulation uses the same secret key to recover the same shared secret.

Why quantum resistant:

Recovering the secret is related to the Module-

LWE problem, which is believed to be hard

for both classical and quantum computers. Shor's

algorithm is not known to efficiently solve Module

LWE.

③ Derive the AES-128 Rijndael key schedule and compute the first two round keys (K_1, K_2) from a given 128-bit cipher key of your choosing, showing the RotWord, SubWord and Rcon steps explicitly.

Ans:

9 minute

Choose the 128-bit cipher key:

$$K_0 = 000102030405060708090A0B0C0D0E0F$$

Split into four 32-bit words:

$$W_0 = 00010203$$

$$W_2 = 08090A0B$$

$$W_1 = 04050607$$

$$W_3 = 0C0D0E0F$$

For AES-128:

$$W_i = W_{i-4} \oplus T(W_{i-1})$$

For every 4th word:

$$T(W) = \text{SubWord}(\text{RotWord}(W)) \oplus \text{Rcon}$$

Round key K_1 :

$$\text{RotWord: } W_3 = 0C0D0E0F$$

$$\text{RotWord}(W_3) = 0D0E0F0C$$

SubWord :

using the AES-S-box:

$$\text{SubWord}(0D0E0F0) = D7AB76FE$$

Rcon :

$$\text{Rcon}_1 = 01000000$$

Therefore,

$$T = D7AB76FE \oplus 01000000 = D6AB76FE$$

Now calculate:

$$W_4 = W_0 \oplus T = 00010203 \oplus D6AB76FE = D6AA74FD$$

$$W_5 = W_1 \oplus W_4 = D2AF72FA$$

$$W_6 = W_2 \oplus W_5 = DAA678F1$$

$$W_7 = W_3 \oplus W_6 = D6AB76FE$$

$$\therefore K_1 = \underline{D6AA74FD \ D2AF72FA \ DAA678F1 \ D6AB76FE}$$

Round Key K2 :

$$\text{RotWord} : \text{RotWord}(W_7) = AB76FE D6$$

$$\text{SubWord} : \text{SubWord}(AB76FED6) = 6238BBF6$$

$$\text{Rcon} : \text{Rcon}_2 = 02000000$$

Therefore

$$T = 6238BBF6 \oplus 02000000 = 6038BBF6$$

$$W_8 = W_4 \oplus T = B692CF0B$$

$$W_9 = W_5 \oplus W_8 = 643DBDF1$$

$$W_{10} = W_6 \oplus W_9 = BE9BC500$$

$$W_{11} = W_7 \oplus W_{10} = 6930B3FE$$

$$\text{Thus, } K_2 = \underline{B692CF0B \ 643DBDF1 \ BE9BC500}$$

$$\underline{6930B3FE}$$

$$W_8 = W_4 \oplus W_4 = 00000000$$

$$W_9 = W_5 \oplus W_5 = 00000000$$

$$W_{10} = W_6 \oplus W_6 = 00000000$$

$$K_1 = \underline{B692CF0B \ 643DBDF1 \ BE9BC500}$$

32 Explain the Mixcolumns transformation as polynomial multiplication over $GF(2^8)$ modulo $x^8 + x^4 + x^3 + x + 1$. Work through the full multiplication of one input column by the fixed Mixcolumn matrix, showing the $GF(2^8)$ arithmetic for each type.

Ans:

8 minute

Each byte of the AES state is an element of $GF(2^8)$ constructed using the irreducible polynomial,

$$m(x) = x^8 + x^4 + x^3 + x + 1 \text{ (hex } 0x11B\text{)}$$

Each column of the state (s_0, s_1, s_2, s_3) is treated as a degree 3-polynomial over $GF(2^8)$,

$$s(x) = s_3x^3 + s_2x^2 + s_1x + s_0$$

Mixcolumns multiplies this by the fixed polynomial

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$

modulo $(x^4 + 1)$ which is equivalent to the matrix form:

$$\begin{bmatrix} S_0' \\ S_1' \\ S_2' \\ S_3' \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} S_0 \\ S_1 \\ S_2 \\ S_3 \end{bmatrix}$$

Worked example:

Taking input column: $S = (d4, bf, 5d, 30)$

$$2 \cdot d4 = \text{time}(d4) = B3$$

$$3 \cdot d4 = 2 \cdot d4 \oplus d4 = B3 \oplus D4 = 67$$

$$2 \cdot bf = \text{time}(bf) = 65$$

$$3 \cdot bf = 65 \oplus BF = DA$$

$$2 \cdot 5d = \text{time}(5d) = BA$$

$$3 \cdot 5d = BA \oplus 5D = E7$$

$$2 \cdot 30 = \text{time}(30) = 60$$

$$3 \cdot 30 = 60 \oplus 30 = 50$$

$$S_0' = 2 \cdot d4 \oplus 3 \cdot bf \oplus 1 \cdot 5d \oplus 1 \cdot 30 = B3 \oplus DA \oplus 5D \oplus 30 = 04$$

$$S_1' = D4 \oplus 65 \oplus E7 \oplus 30 = 66$$

$$S_2' = D4 \oplus BF \oplus BA \oplus 5D = 81$$

$$S_3' = 67 \oplus BF \oplus 5D \oplus 60 = E5$$

∴ Output column = (04, 66, 81, E5)

33) Critically Evaluate AES's resistance to side channel attacks, specifically timing analysis and differential power analysis. Describe at least two concrete countermeasures (eg: constant time, masking) and their performance trade-off.

Ans:

6 minute

AES is cryptographically strong, but a practical implementation can leak information through its physical behaviour.

1. Timing / cache attacks:

Table based AES implementations use lookup tables for the S-box. The memory access pattern depends on the secret key and input data, causing different cache hit / miss. An attacker can measure these

timing differences and recover key info.

2. Differential Power Analysis

AES operations consume slightly diff amount of power depending on intermediate values.

By collecting many power traces an attacker can infer secret key bits.

Countermeasures:

i) Constant time implementation:

Use a constant time S-box so execution time and memory access do not depend on secret data.

- Trade-off: many require more computation and can be slower than optimized table lookups.

ii) Masking:

Randomly mask sensitive intermediate values, such as: $x' = x \oplus r$

where r is random mask.

Trade off: Adds extra operations, randomness generation and implementation complexity.

34) Using the partial AES-S box below, perform the subBytes transformation on the state word, then apply Shift Rows to a 4x4 state matrix that has this first row.

Row\col→	3	4	7	9	A	C
1	6D			C6		
2	04					
4		A1				2E
A			63		D2	

Ans:

7 minute

subBytes using S-box:

Input Byte	High nibble (row)	Low nibble (column)	S-box output
0x23	2	3	04
0xA7	A	7	63
0x4C	4	C	2E
0x19	1	9	C6

After subBytes : [D4, 63, 2E, C6]

Building 4x4 state matrix:

D4	63	2E	C6
00	00	00	00
00	00	00	00
00	00	00	00

Applying shift rows:

Row 0 → no shift

Row 1 → shift left by 1

Row 2 → shift left by 2

Row 3 → shift left by 3

no visible
effect

Final matrix after shift rows:

D4	63	2E	C6
00	00	00	00
00	00	00	00
00	00	00	00

③5 Compare the error propagation and parallelizability characteristics of ECB, CBC, CFB, OFB and CTR modes. using a labelled diagram for each. Recommend the most suitable for a) encrypting a live video stream over a lossy network and b) encrypting a static disk image with justification.

Ans:

10 minute

Comparison of ECB, CBC, CFB, OFB and CTR modes:

1. ECB:

Diagram:

$$P_1 \rightarrow E(K) \rightarrow C_1$$

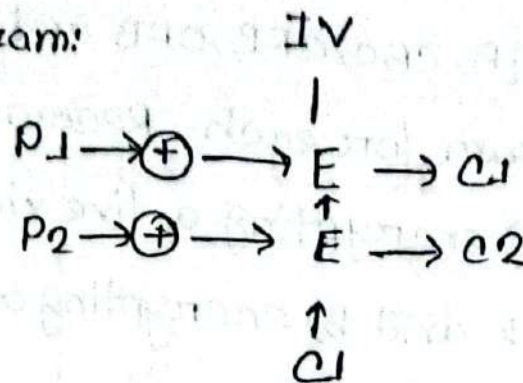
$$P_2 \rightarrow E(K) \rightarrow C_2$$

$$P_3 \rightarrow E(K) \rightarrow C_3$$

- Parallel Encryption and decryption is possible
- A bit error affects only the corresponding block.
- Identical plaintext blocks creates identical ciphertext blocks, so patterns are revealed $\rightarrow$ it is the main problem.
- Unique IV: No

2. CBC:

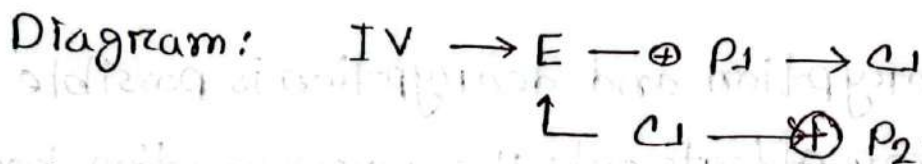
Diagram:



- Parallel encryption: No
- Parallel decryption: Yes
- Error: A bit error in C_i corrupts the current plaintext block and affects the corresponding bit of the next block.
- Unique IV: Yes

3. CFB:

Diagram:



- Parallel encryption: No
- Parallel decryption: Yes
- Error: A ciphertext bit error affects the current block and the next block.
- Unique IV: Yes

4. OFB:

Diagram: $IV \rightarrow E \rightarrow S_1 \oplus P_1 \rightarrow C_1$

$\downarrow$
 $E \rightarrow S_2 \oplus P_2 \rightarrow C_2$

- Parallel encryption: No
- Parallel decryption: No
- Error: A bit error affects only the corresponding bit
- Unique IV: Yes and never reused.

5. CTR:

Nonce₁ $\rightarrow E \rightarrow S_1 \oplus P_1 \rightarrow C_1$

Nonce₂ $\rightarrow E \rightarrow S_2 \oplus P_2 \rightarrow C_2$

Nonce₃ $\rightarrow E \rightarrow S_3 \oplus P_3 \rightarrow C_3$

- Parallel encryption: Yes
- Parallel decryption: No
- Error: A bit error affects only the corresponding bit
- Unique IV/Nonce: Unique and not reused.

a) Live video stream over a lossy network:

CTR is most suitable because a transmission error only affect the corresponding block and does not propagate to other blocks.

b) static disk image:

CTR is suitable because encryption and decryption can be done in parallel and an error in one block does not corrupt other blocks. It also allows efficient random access to blocks.

36 Critically assess a lightweight cryptographic standard suitable for resource constrained IoT devices against AES-128 in terms of memory footprint, energy consumption and side channel resistance. Recommend which is better for a battery powered sensor with 40 KB of RAM.

Ans:

6 minute

Ascon Vs AES-128 for IoT:

Factor	Ascon	AES-128
Memory footprint	Small, designed for constrained devices	Usually larger, especially without hardware AES.
Energy consumption	Generally lower on small microcontrollers	Can be higher in software
Side channel resistance	Designed with lightweight implementations	Strong security possible
IoT suitability	Specially suitable for constrained IoT devices	Designed as a general purpose cipher

Assessment:

Aeson provides authenticated encryption with a small state and is optimized for resource constrained devices. AES-128 is very secure and widely supported, but software implementations can require more memory. Hardware

AES can reduce this disadvantage.

Recommendation:

For a battery powered sensor with less than 10 KB RAM, I would choose Aeson, especially when AES hardware acceleration is unavailable. Because it is lightweight and can reduce memory usage and computational cost.

37) Design a bandwidth and duty cycle budget for an IoT sensor network in which each node transmits a 1200 byte packet once per minute over a LPWAN technology of your choosing. Compute the total data volume per node per day and discuss regulatory duty-cycle constraints that could affect the design.

Ans:

7 minute

Assume LoRaWAN EU868, and one node sends 1200 byte packet every minute.

Data volume per node:

Packets per day : $24 \times 60 = 1440$ packets/day

Daily data : $1200 \times 1440 = 1278000$ bytes/day

$$= \frac{1278000}{1024} \text{ KB/day}$$

$$= 1687.5 \text{ KB/day}$$

$\therefore$ So each node generates 1687.5 KB/day of application data.

Duty cycle problem:

A 1% duty cycle allows a transmitter to occupy the channel for at most

$$24 \times 3600 \times 0.01 = 864 \text{ seconds/day}$$

③⑧ Explain why a plain hash-based MAC is vulnerable to a length extension attack and derive how HMAC's nested construction $H(K \oplus \text{opad}, 1, H(K \oplus \text{ipad}, 1, M))$ eliminates this vulnerability. Illustrate with a small worked MAC examples.

Ans:

Length explain Attack on $H(K || M)$

A Merkle Damgaard has works as a chain:

$$H_i = f(H_{i-1}, M_i)$$

If a MAC is simplify:

$$\text{MAC} = H(K || M)$$

the final hash values acts like the internal chaining state. If an attacker knows MAC and the length of $K||M$, they can continue the hash computation with a new message X :

$$H(K||M||\text{Pad}||X)$$

Without knowing K . Thus an attacker can create a valid MAC for an extended message.

Simple example:

Suppose a simplified hash is :

$$H(x_1, x_2) = x_1 + x_2 \pmod{100}$$

Let, $K = 20, M = 30$

$$\therefore \text{MAC} = H(20||30) = 20 + 30 = 50$$

If the attacker wants to append $X = 10$, they can continue from the known state $= 50 + 10 = 60$

So attacker can construct a valid MAC for extended message without knowing K .

How HMAC fixes H:

HMAC used nested hashing:

$$H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M))$$

Here the inner hash:

$$I = H((K \oplus \text{ipad}) \parallel M)$$

outer hash:

$$\text{MAC} = H((K \oplus \text{opad}) \parallel I)$$

The attacker may try to extend the inner hash
but the result is only an output input to the
outer hash. They do not know $K \oplus \text{opad}$, so they
can't generate a valid outer MAC for the
modified inner result.

39) Diagram The TLS 1.3 handshake (client Hello with key shares, encrypted extensions, finished message) and explain how the session key are divided via HKDF from the DHE shared secret. Compare this with 1.2 TLS handshake in terms to round trip count and forward secrecy.

Ans:

7 minute

TLS 1.3 handshake:

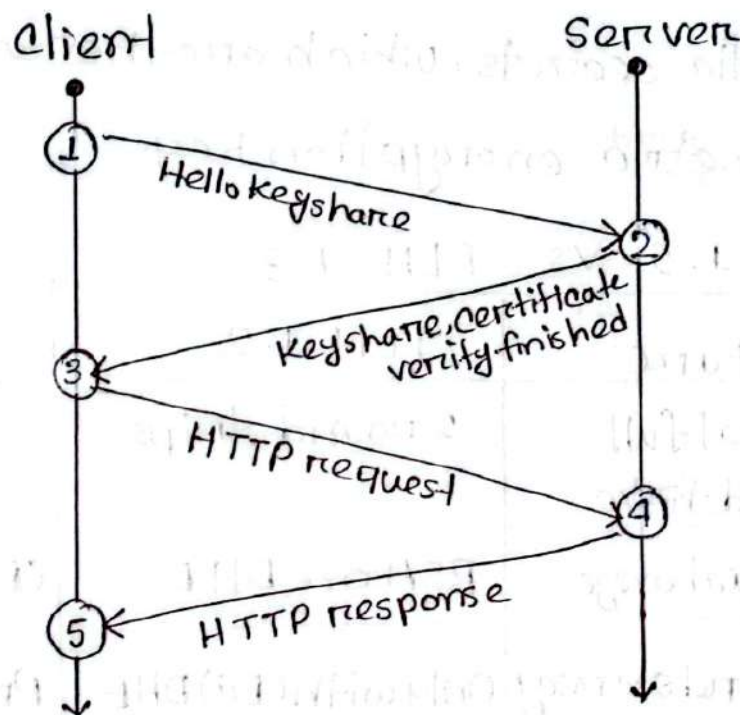


Diagram of TLS 1.3 handshake

Key derivation using HKDF:

Both sides calculate the same DHE shared secret:

$$Z = E_{DH}(\text{private}, \text{peer-public})$$

TLS 1.3 uses HKDF to deliver the traffic keys -

Early secret $\rightarrow$ Handshake secret $\rightarrow$ Master secret

The DHE secret is mixed into the Handshake secret. From the resulting secrets, HKDF derives separate server/client handshake and application traffic secrets, which are then used to generate symmetric encryption keys.

TLS 1.2 vs TLS 1.3

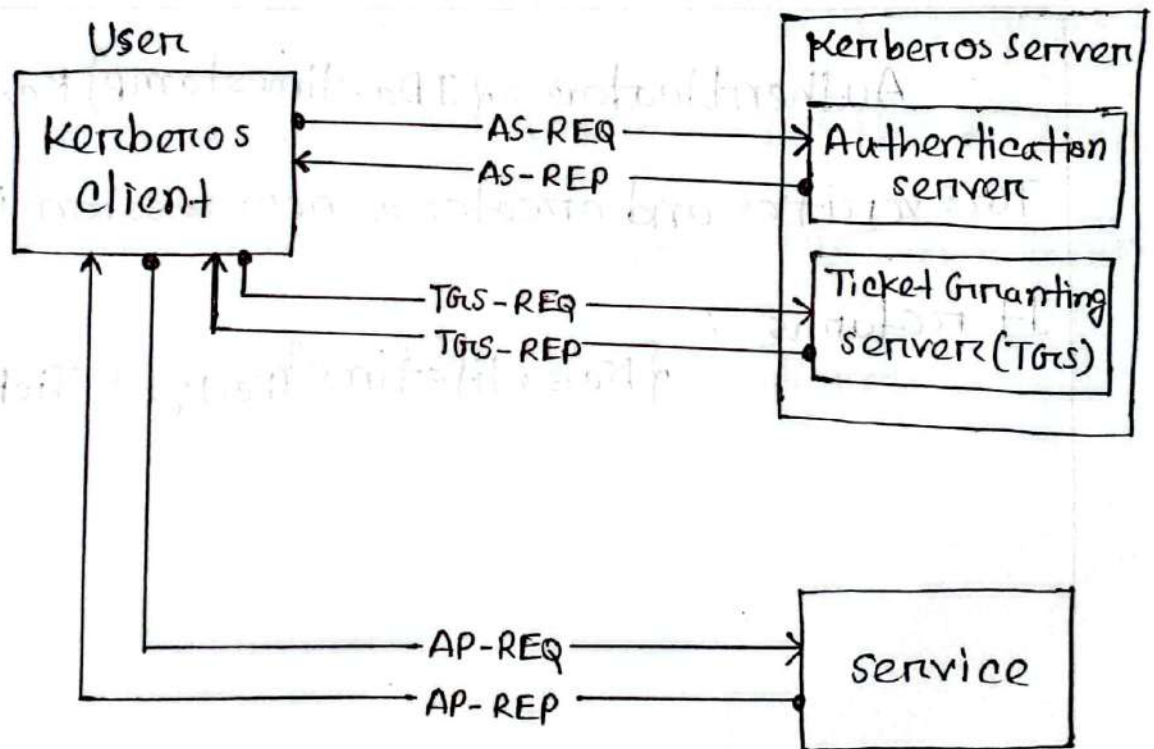
Feature	TLS 1.2	TLS 1.3
Typical full handshake	2 round trips	1 round trip
Key exchange	RSA or DHE	(EC)DHE
Forward secrecy	Only with (EC)DHE	Provided by (EC)DHE
Encryption starts	Later in handshake	Earlier

④ Diagram and explain the full Kerberos authentication protocol, covering the Authentication server exchange, Ticket Granting Server exchange and client server exchange. Include how session keys and ticket lifetimes are established and why the TGT avoids repeated password transmission.

Ans:

9 minute

Kerberos uses a trusted authentication server and ticket granting server so the user's password is not repeatedly sent.



1. AS exchange:

Client sends its identity ID_c and requests a TGT.

AS generates a session key $K_{c,as}$ and sends:

$$\{K_{c,as}, TGT\}_{K_c}$$

where K_c is derived from the user's password.

The TGT contains:

$$TGT = \{ID_c, lifetime, K_{c,as}\}_{K_{as}}$$

2. TGS exchange:

Client sends TGT and an Authentication.

$$Authenticator = \{ID_c, timestamp\}_{K_{c,as}}$$

TGS verifies and creates a new session key $K_{c,s}$

It returns:

$$\{K_{c,s}, lifetime\}_{K_{c,as}} + Tickets$$

3. Time

3. client server exchange:

client sends Ticket_s and a new authenticator
to the service server.

The server decrypts the ticket, obtains K_c and verifies the authenticator. This provides mutual authentication.

Why TGT avoids repeated password transmission:

The password is never sent on the network. It is used to obtain K_c and decrypt the AS response. After that, the client uses the TGT and temporary session keys to obtain service tickets.